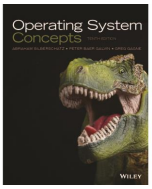


Operating Systems 2025

U3680

National Taipei University

Final Exam [Chapter 6 - Chapter 10]

1. Answer the following questions about synchronization:

   (a) Provide a brief explanation of the **progress** and **bounded waiting** properties. (8%)

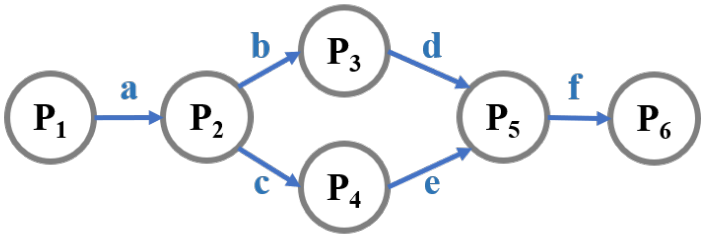
   - Progress: if no process is executing in its CS and there exist some processes that wish to enter their CS, the processes cannot be postponed indefinitely. (4%)
   - Bounded waiting: a bound must exist on the number of times that other processes are allowed to enter their CS after a process has made a request to enter its CS. (4%)

   (b) Complete the table below by marking **"yes"** if the method satisfies the progress or bounded waiting properties, and **"no"** if it does not. (8%)

	Peterson's solution	Bakery algorithm	Atomic TestAndSet	Atomic Swap
progress	yes	yes	yes	yes
bounded waiting	yes	yes	no	no

2. In the below figure, each edge represents a dependency between two processes (*e.g.*, $P_1 \rightarrow P_2$ means P_1 must complete before P_2 starts). Please complete the required operations for each process using **wait**, the **operation** S_i **for process** P_i, and **signal** with the **semaphore variables shown on the edges** (6%)

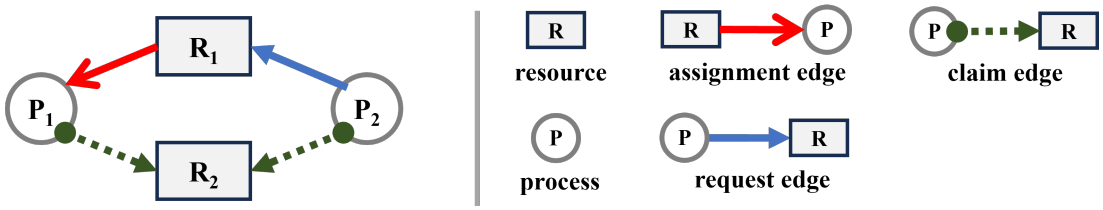
P_1: **S_1; signal(a);**
P_2: **wait(a); S_2; signal(b); signal(c);**
P_3: **wait(b); S_3; signal(d);**
P_4: **wait(c); S_4; signal(e);**
P_5: **wait(d); wait(e); S_5; signal(f);**
P_6: **wait(f); S_6;**



3. List and briefly describe the **four necessary conditions** required for a **deadlock** to occur. (8%)

   - Mutual exclusion: only 1 process at a time can use a resource (2%)
   - Hold and wait: a process holding some resources and is waiting for another resource (2%)
   - No preemption: a resource can be only released by a process voluntarily (2%)
   - Circular wait: There exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \cdots \rightarrow P_n \rightarrow P_0$ (2%)

4. Based on the following resource-allocation graph for **deadlock avoidance**, should the OS grant (accept) P_2's requests for R_2, Explain your reasoning. (4%)



No. If the OS allocates R_2 to P_2, a cycle will form: $P_2 \rightarrow R_1 \rightarrow P_1 \rightarrow R_2$ due to the claim edge, leading to a deadlock when P_1 requests R_2.

5. In a system, many programs may use the same system libraries (*e.g.*, the C library). Can we use **dynamic loading** to avoid loading multiple copies of the same library into memory? Explain why or why not. (4%)
No. because dynamic loading only allows parts of the code or library **within a program** to be loaded as needed. As a result, using dynamic loading for certain functions may still result in multiple copies residing in memory when each process invokes them. In this case, we should use dynamic linking.

6. Consider the following snapshot of a system. The total resource (A, B, C, D) = (4, 3, 4, 3). Using **Banker's algorithm** to answer the following questions:

   (a) Is the current state safe? If so, specify the sequence in which the processes can be executed to completion. If not, provide a reason. (4%)
   Yes. There exists a safe sequence: $T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$

   (b) If process P_1 requests an additional instance of resource A, should the OS grant the allocation? Explain your answer. (4%)
   Yes. There exists a safe sequence: $T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$

	Allocation				Max			
	A	B	C	D	A	B	C	D
P_1	1	1	2	1	4	2	3	2
P_2	1	0	0	1	1	0	1	1
P_3	0	1	1	0	0	1	1	1
P_4	1	1	0	1	1	2	0	1

7. Answer the following questions about **swapping**:

   - When swapping a process back into memory, is it possible to place it at a different memory location than its original one? **Explain** your answers based on execution-time, compile-time, and load-time address binding. (6%)
   In compile-time and load-time address binding, the process must be swapped back into the same memory location because physical addresses are fixed within the code loaded into memory. In contrast, with execution-time address binding, the process can be swapped back into a different location since the logical addresses in the code are translated to physical addresses at runtime by the MMU.
   - Explain why a process that is performing I/O operations cannot be swapped out. (4%)
   Performing I/O involves transferring data from disk to memory. If the process responsible for the I/O has been swapped out, its allocated memory is no longer valid.

8. Assume a system uses **16 bits** logical address and **1 KB** page size. A **single-level page table** is used by the MMU. The physical memory is **1 MB**. Please answer the following questions:

   (a) What is the **logical address** for **page number 17** and **offset 63**? (4%)
   0100010000111111

   (b) Assume that **page 17** maps to **frame 24**, what is the **physical address** in (a)? (4%)
   00000110000000111111

9. Assume each memory read takes **100** nanoseconds and each TLB search takes **10** nanoseconds in a **single-level paging** system. What is the necessary TLB hit rate if we need the effective memory reference time smaller than **130** nanoseconds? (6%)
   80%

10. Why is **hierarchical paging** challenging to implement in 64-bit systems? **Please explain the reason in detail.** (4%)
    Hierarchical paging for 64-bit logical addresses faces a dilemma: fewer levels lead to a large page table, whereas more levels increase the number of memory accesses.

11. Provide one advantage and one disadvantage of **inverted page table** (frame table) (4%)

    - Advantages: (1) there is no requirement to allocate and maintain page tables for every process; (2) only a single frame table needs to be managed for the entire system, and its memory allocation can remain fixed.
    - Disadvantages: (1) it is necessary to store both the PID and the page number.; (2) each access needs to search the whole frame table; (3) hard to support shared page/memory.

    Note: any pair of answers is acceptable.

12. Apply the **Optimal** and the **Least Recently Used (LRU)** algorithms for page replacement with the page-reference strings: **(1, 2, 3, 4, 1, 2, 3, 4, 1)**. Find the number of page faults for each algorithm, assuming demand paging with **three** frames. **You need to show the progress of each step.** (8%)

    The Optimal algorithm results in 5 page faults, whereas the LRU algorithm results in 9.

13. Explain how the **Working-Set Model** is used to determine if a system has thrashing. (6%)
    The Working-Set Model tracks the set of **referenced pages** by each process within a defined time window, known as the working-set size. If the combined working-set sizes of all processes exceed the number of available frames, a process is suspended to prevent thrashing.

14. Explain how the **Page-Fault Frequency** approach helps prevent thrashing. Is it more efficient than the **Working-Set Model**? Justify your answer. (8%)
The Page-Fault Frequency method sets upper and lower bounds for a process's acceptable page-fault rate. If the rate exceeds the upper limit, an additional frame is allocated to the process; if it falls below the lower bound, a frame is removed. (4%)
The Page Fault Frequency method is more efficient than the Working-Set Model because it updates only when a page fault occurs, which happens **less frequently**. In contrast, the Working-Set Model must track **every page reference for each process**, making it more resource-intensive. (4%)